

DOCUMENTATION



Your Company

Fern Docs Starter

This is a custom subtitle!

February 16, 2026

Table of Contents

Docs

GET STARTED

Welcome	1
Edit your docs	4
Write content	6
Set up navigation	8
Customize your docs	10
Support	12

CHANGELOG

Changelog

2025

October 1900

October 29, 2025	13
October 15, 2025	14
October 1, 2025	15

September 1900

September 15, 2025	16
September 1, 2025	17

API Reference

OVERVIEW

API reference	18
---------------------	----

PLANT STORE API

Plants

Add a new plant to the store	21
Update an existing plant	23
Search plants by status	25
Search plants by tags	27
Find plant by ID	29

Users

Logs user into the system	30
---------------------------------	----

Logs out current logged-in user session	31
Get user by username	32
Plant Events	
plantEvents	33

Get started

Welcome to Fern!

Everything you need to build the best developer experience

This repository is a starter template for building documentation with [Fern](#). It includes a sample Plant Store API to demonstrate how Fern generates interactive API reference documentation from an OpenAPI specification.

Use this template to get started quickly, then replace the sample content with your own API and documentation.

What's included

This starter template contains everything you need to publish documentation:

```
📄 docs.yml # Configures navigation, theme, and hosting
📄 fern.config.json # Sets your organization name and CLI version
📄 openapi.yaml # Sample Plant Store API specification
▼ 📁 docs
  > 📁 pages # Markdown content for your documentation
  > 📁 assets # Logos, favicon, and other assets
```

Getting started

1 Clone this repository

```
$ git clone <your-template-repo.git>
$ cd docs-starter
```

2 Install the Fern CLI

```
$ npm install -g fern-api
```

3 Preview locally

Start a local development server with hot-reloading:

```
$ fern docs dev
```

Open <http://localhost:3000> to see your docs.

4

Make it yours

Update `fern.config.json` with your organization name and `docs.yml` with your desired URL. The `version` refers to [the Fern CLI](#):

`fern.config.json`

```
1  {
2    "organization": "your-org-name",
3    "version": "0.46.15"
4  }
```

`docs.yml`

```
1  instances:
2    - url: your-org.docs.buildwithfern.com
```

5

Publish

When you're ready to go live:

```
$ fern generate --docs
```

Next steps

Replace the sample Plant Store API with your own OpenAPI specification, update the Markdown pages with your content, and customize the theme to match your brand. For detailed guidance, explore the pages in this documentation or visit the [Fern documentation](#).

Edit your docs

Learn how to preview and publish changes using the CLI or Fern Editor

Set up navigation

Configure tabs, sections, and pages in your docs.yml file

Customize your docs

Brand your documentation with colors, logos, and layouts

API reference

See how Fern generates interactive API documentation

[Get started](#)

Edit your docs

Preview changes locally and publish updates using the CLI, Fern Editor, or Fern Writer

Fern provides three ways to edit your documentation: the command-line interface (CLI) for developers, Fern Editor for team members who prefer a visual interface, and Fern Writer for AI-assisted documentation updates via Slack.

▼ Using the CLI

The Fern CLI lets you preview changes locally and publish updates from your terminal.

1 Install the CLI

```
$ npm install -g fern-api
```

2 Preview locally

Start a local development server with hot-reloading:

```
$ fern docs dev
```

This serves your documentation at <http://localhost:3000> [↗](#).

3 Generate a preview link

Create a shareable preview URL for your team to review changes before publishing:

```
$ fern generate --docs --preview
```

4 Publish to production

When you're ready for your docs to be publicly accessible:

```
$ fern generate --docs
```

This builds and publishes your documentation to the URL configured in `docs.yml`.

For detailed CLI documentation, see the [Fern CLI reference](#).

> **Using Fern Editor**

> **Using Fern Writer**

Next steps

Write content

Create pages using MDX and components

Set up navigation

Configure tabs, sections, and pages

Customize your docs

Brand your documentation

Get started

Write content

Create documentation pages using MDX

Documentation pages are written in MDX, which combines Markdown with React components. Create `.mdx` files in the `docs/pages/` directory and reference them in your `docs.yml` navigation.

Basic page structure

docs/pages/my-page.mdx

```
1  ---
2  title: Page title
3  subtitle: Optional subtitle
4  slug: my-page
5  ---
6
7  Your content here. Use standard Markdown formatting like bold, italic, and
   `code`.
8
9  ## Headings
10
11 Use headings to organize your content.
```

Using components

Fern provides [built-in components](#) for common documentation patterns:

 Highlight important information with callouts.

First card
Card description

Second card
Another description

Markdown

```
1  <Note>
2  Highlight important information with callouts.
3  </Note>
4
5  <CardGroup cols={2}>
6    <Card title="First card" href="/page-one">
7      Card description
8    </Card>
9    <Card title="Second card" href="/page-two">
10     Another description
11  </Card>
12 </CardGroup>
```

Get started

Set up navigation

Configure tabs, sections, and pages in your docs.yml file

The `docs.yml` file controls the navigation structure of your documentation site. Explore the navigation options below and click through to the Fern documentation for detailed configuration guides.

Sections and pages

Organize content into sections with pages and folders

Tabs

Create top-level navigation groups

Versions

Add version switching for your documentation

Products

Support multiple products in one site

API reference

Auto-generate API documentation from your spec

Changelogs

Add a changelog section to your docs

Quick example

Here's a minimal example showing navigation structure in `docs.yml`:

docs.yml

```
1  navigation:
2    - section: Getting started
3      contents:
4        - page: Introduction
5          path: docs/pages/intro.mdx
6        - page: Quickstart
7          path: docs/pages/quickstart.mdx
8    - api: API Reference
```

For complete navigation options, see the [navigation documentation](#) .

Get started

Customize your docs

Brand your documentation with colors, logos, and layouts

The `docs.yml` file controls the visual appearance of your documentation site. Explore the customization options below and click through to the Fern documentation for detailed configuration guides.

Colors

Set brand colors for light and dark modes

Logo and favicon

Add your logo and favicon

Typography

Customize fonts for headings and body text

Layout

Configure page width, tabs placement, and more

Navbar links

Add links and buttons to your navigation bar

Custom CSS

Advanced styling with custom CSS and JavaScript

Quick example

Here's a minimal example showing common customization options in `docs.yml`:

docs.yml

```
1  title: Your Documentation
2
3  colors:
4    accentPrimary:
5      dark: "#81C784"
6      light: "#1B5E20"
7
8  logo:
9    dark: docs/assets/logo-dark.svg
10   light: docs/assets/logo-light.svg
11
12  favicon: docs/assets/favicon.svg
```

For complete configuration options, see the [site-level settings documentation](#) .

[Get started](#)

Support

Get help from the Fern team and community

Have questions about Fern or need help with your documentation? Start here.

Contact Fern

For product questions, migrations, or enterprise support, reach out to the Fern team:

- Join the [Fern Slack community](#)  for quick questions and feedback
- Use the [contact form](#)  for sales, enterprise, or migration inquiries

GitHub and issues

If you run into bugs or want to request features, open an issue on GitHub:

- [docs-starter repository](#)  – issues specific to this starter template
- [Fern repository](#)  – general Fern platform bugs and feature requests

[Changelog](#) > [Changelog](#) > [2025](#) > [October 1900](#)

October 29, 2025

Enhancements

Inventory tracking improvements

Added new `reserved` field to inventory responses, allowing better tracking of stock allocated to pending orders.

Bulk operations

Introduced `/inventory/batch-update` endpoint for updating multiple plant inventory levels in a single request.

Bug fixes

- Fixed an issue where low stock alerts weren't triggering when inventory reached exactly the reorder point
- Corrected timezone handling in inventory history timestamps

[Changelog](#) > [Changelog](#) > [2025](#) > [October 1900](#)

October 15, 2025

New features

Webhook events

Added support for new webhook events:

- `inventory.low_stock` - Triggered when stock falls below reorder point
- `inventory.restocked` - Triggered when items are restocked
- `customer.updated` - Triggered when customer profile is modified

Multi-location inventory

Inventory endpoints now support `location` parameter for tracking stock across multiple warehouses and greenhouses.

Enhancements

- Improved webhook retry logic with exponential backoff
- Added `tracking_number` and `carrier` fields to order responses

[Changelog](#) > [Changelog](#) > [2025](#) > [October 1900](#)

October 1, 2025

New features

Customer loyalty program

Introduced new `/customers/{id}/loyalty` endpoint for accessing customer loyalty points and rewards.

Wishlist functionality

Customers can now maintain wishlists:

- `POST /customers/{id}/wishlist` - Add plants to wishlist
- `GET /customers/{id}/wishlist` - Retrieve wishlist
- `DELETE /customers/{id}/wishlist/{plant_id}` - Remove from wishlist

Breaking changes

Order status field

The order `status` field now uses standardized values. Legacy status values will be deprecated on December 1, 2025:

- `pending_payment` → `pending`
- `in_progress` → `processing`

Please update your integrations to use the new values.

Enhancements

- Added pagination support to `/plants` endpoint with `limit` and `offset` parameters
- Improved error messages with more detailed validation feedback

[Changelog](#) > [Changelog](#) > [2025](#) > [September 1900](#)

September 15, 2025

New features

Plant care requirements

Added detailed care information to plant objects:

- `light_requirements` - Light level needs (low, medium, bright indirect, full sun)
- `water_frequency` - Watering schedule recommendations
- `temperature_range` - Ideal temperature range
- `humidity_level` - Humidity preferences

Advanced filtering

Plant search now supports filtering by care requirements:

```
$ GET /plants?care_level=easy&light_requirements=low
```

Enhancements

- Increased rate limit from 1000 to 5000 requests per hour for production API keys
- Added response caching headers to improve performance
- Order confirmation emails now include estimated delivery date

Bug fixes

- Fixed pagination issue in customer order history
- Resolved incorrect stock calculations when orders were cancelled

[Changelog](#) > [Changelog](#) > [2025](#) > [September 1900](#)

September 1, 2025

New features

Order refunds API

Introduced comprehensive refund management:

- `POST /orders/{id}/refund` - Process full or partial refunds
- `GET /orders/{id}/refunds` - View refund history
- Automatic inventory restoration on refunded items

Webhook signature verification

All webhooks now include `x-plantstore-signature` header for verifying webhook authenticity using HMAC-SHA256.

Enhancements

- Added `description` field to plant objects for detailed product information
- Customer addresses can now be labeled (e.g., “Home”, “Work”, “Gift Address”)
- Improved API response times by 40% through database optimization

Documentation

- Added new tutorial: “Managing plant inventory”
- Updated webhook setup guide with signature verification examples
- Added code samples for Python, JavaScript, and Go SDKs

[Overview](#)

API reference

Interactive documentation generated from your API specification

Fern automatically generates interactive API reference documentation from your API specification. This starter template includes a sample Plant Store API to demonstrate the feature.

Supported specifications

Fern supports multiple API specification formats:

Format	Description	Documentation
OpenAPI	The most widely used REST API specification format (versions 2.0, 3.0, and 3.1)	OpenAPI docs 
AsyncAPI	For event-driven and message-based APIs	AsyncAPI docs 
gRPC	Protocol buffer definitions for gRPC services	gRPC docs 

How it works

Place your API specification file in the `fern/` directory and reference it in your `docs.yml` navigation:

`docs.yml`

```
1  navigation:
2    - section: Documentation
3      contents:
4        - page: Overview
5          path: docs/pages/overview.mdx
6    - api: API Reference
```

Fern parses your specification and generates:

- Endpoint documentation with request/response schemas
- Interactive API Explorer for testing endpoints
- Code snippets in multiple languages
- Type definitions and examples

API Explorer

The API Explorer lets users test API endpoints directly in the documentation. Users can fill in parameters, send requests, and see responses without leaving the page.

 For advanced API Explorer features like auto-populating API keys, see the [API Explorer documentation](#).

Customizing the API reference

Control how your API reference appears using the `api` configuration in `docs.yml`:

`docs.yml`

```
1 navigation:
2   - api: API Reference
3     api-name: plantstore
4     paginated: true
```

You can also add custom content to API reference pages, organize endpoints into groups, customize section names, and control which endpoints are visible.

 For detailed customization options, see the [API reference documentation](#).

SDK code snippets

If you generate SDKs with Fern, code snippets automatically appear in your API reference showing how to call each endpoint using your SDK:

`generators.yml`

```
1 groups:
2   python-sdk:
3     generators:
4       - name: fernapi/fern-python-sdk
5         version: 4.3.1
6         output:
7           location: pypi
8           package-name: plantstore
```

 For more information on SDK generation, see the [SDK documentation](#).

Add a new plant to the store

Add a Fern Plant

Successful Plant Addition

POST /v3/plant

```
1 curl -X POST https://api.plantstore.dev/v3/plant \  
2     -H "Content-Type: application/json" \  
3     -d '{  
4     "name": "Fern",  
5     "category": "Indoor",  
6     "tags": [  
7         "green",  
8         "leafy"  
9     ],  
10    "status": "available"  
11  }'
```

```
1  {  
2    "id": 101,  
3    "name": "Fern",  
4    "status": "available",  
5    "tags": [  
6        "green",  
7        "leafy"  
8    ]  
9  }
```

Request

Details of the plant to add

name string Optional

category string Optional

tags list of strings Optional

status enum Optional

Allowed values: **available** **pending** **sold**

Response

id integer or null

name string or null

status string or null

tags list of strings or null

Errors

405

Method Not Allowed Error

Update an existing plant

Update plant status

Successful Plant Update

PUT /v3/plant

```
1 curl -X PUT https://api.plantstore.dev/v3/plant \  
2     -H "Content-Type: application/json" \  
3     -d '{  
4     "name": "Fern",  
5     "category": "Indoor",  
6     "tags": [  
7         "green",  
8         "leafy"  
9     ],  
10    "status": "sold"  
11  }'
```

```
1  {  
2    "id": 101,  
3    "name": "Fern",  
4    "status": "sold",  
5    "tags": [  
6        "green",  
7        "leafy"  
8    ]  
9  }
```

Request

Updated details of the plant

name string Optional

category string Optional

tags list of strings Optional

status enum Optional

Allowed values: **available** **pending** **sold**

Response

id integer or null

name string or null

status string or null

tags list of strings or null

Errors

400	Bad Request Error
404	Not Found Error

Search plants by status

GET

/v3/plant/search/status

1

curl https://api.plantstore.dev/v3/plant/search/status

200

Plants with status available

1

[

2

{

3

"id": 101,

4

"name": "Fern",

5

"status": "available",

6

"tags": [

7

"green",

8

"leafy"

9

]

10

},

11

{

12

"id": 102,

13

"name": "Palm",

14

"status": "available",

15

"tags": [

16

"tropical"

17

]

18

}

19

]

Filter plants based on their current status.

Query parameters

status enum Optional

The status of plants to search for.

Allowed values: available pending sold

Response

List of plants matching the status filter

id ⓘ integer or null

name ⓘ string or null

status ⓘ string or null

tags ⓘ list of strings or null

Search plants by tags

GET

/v3/plant/search/tags

1 curl https://api.plantstore.dev/v3/plant/search/tags

200

Plants filtered by tags

1 [
2 {
3 "id": 101,
4 "name": "Fern",
5 "status": "available",
6 "tags": [
7 "green",
8 "leafy"
9]
10 },
11 {
12 "id": 103,
13 "name": "Cactus",
14 "status": "available",
15 "tags": [
16 "spiky",
17 "desert"
18]
19 }
20]

Filter plants based on associated tags.

Query parameters

tags list of strings Optional
Tags to filter plants (comma-separated).

Response

List of plants matching the tags filter

id ⓘ integer or null

name ⓘ string or null

status ⓘ string or null

tags ⓘ list of strings or null

Find plant by ID

GET

/v3/plant/:plantId

```
1 curl https://api.plantstore.dev/v3/plant/1
```

200

Plant details

```
1 {
2   "id": 101,
3   "name": "Fern",
4   "status": "available",
5   "tags": [
6     "green",
7     "leafy"
8   ]
9 }
```

Retrieve a plant's details by its ID.

Path parameters

plantId integer **Required**
ID of the plant to retrieve

Response

Details of the requested plant

id integer or null

name string or null

status string or null

tags list of strings or null

Logs user into the system

GET /v3/user/auth/login

1 curl https://api.plantstore.dev/v3/user/auth/login

200 Successful login response

1 {
2 "token": "abc123token",
3 "expiresIn": 3600
4 }

Query parameters

username string Optional

The username for login

password string Optional

The password for login

Response headers

Set-Cookie string or null

Session token for user authentication

Response

Successful login

token string or null

Authentication token

expiresIn integer or null

Token expiration time in seconds

Logs out current logged-in user session

GET /v3/user/auth/logout
1 curl https://api.plantstore.dev/v3/user/auth/logout

Get user by username

GET

/v3/user/:username

```
1 curl https://api.plantstore.dev/v3/user/username \  
2     -H "Content-Type: application/json"
```

200

User details response

```
1 {  
2   "id": 42,  
3   "username": "green_thumb99",  
4   "email": "green.thumb99@plantmail.com"  
5 }
```

Retrieve user details using their username.

Path parameters

username string Required

Username of the user to retrieve

Response

User details retrieved successfully

id integer or null

username string or null

email string or null

plantEvents

Handshake	
URL	wss://ws.plantstore.dev/ws/plants
Method	GET
Status	101 Switching Protocols
Messages	
↑ { "action": "subscribe" }	subscribe ▾
↓ { "type": "plant.created", "occurredAt": "2024..." }	receivePlantEvents ▾

Connect to receive real-time plant lifecycle events.
Subscribe to specific event types or receive all events.

WSS wss://ws.plantstore.dev/ws/plants

Headers

Authorization string **Required**
Bearer token for authentication.

Query parameters

eventTypes string **Required**
Comma-separated list of event types to subscribe to.
Valid values: plant.created, plant.updated, plant.status.changed
If omitted, all events are delivered.

plantId string **Required**

Send

↑

subscribe

object

Required

Send a subscription request to filter events.

+ Show 3 properties

Receive

↓

receivePlantEvents

object

Required

Receive plant lifecycle events from the server.

+ Show 3 properties

OR

receivePlantEvents

object

Required

Receive plant lifecycle events from the server.

+ Show 3 properties

OR

receivePlantEvents

object

Required

Receive plant lifecycle events from the server.

+ Show 5 properties